

Aula 3 — Flutter a fundo: UI, Estado & Isolates

"Flutter vence o React Native?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 17/06/2026

Recap da Aula 2

- Old vs New Architecture (JSI/Fabric/TurboModules/Hermes)
- React Navigation v7, RTK Query, Reanimated worklets
- App de feed com cache, animação e MMKV

Atividade 2 — entregue?

Agenda da aula (3h30)

1. **Bloco 1 (75min)** — Flutter no **DartPad** (zero install): Dart, widgets, **hands-on do MovieCard, estado (Riverpod)**
2. **Setup + Intervalo (30min)** — começa o download do **Android Studio + Flutter**; volta com tudo baixado
3. **Bloco 2 (75min)** — no **Android Studio**: roda o projeto + começa a Atividade 3 (Ex1/Ex2) · Skia/Impeller · comparativo RN × Flutter · isolates
4. **Esquenta (15min)** — Atividade 3 (continua em casa) + prévia Aula 4 (KMP + bridging)

 Bloco 1 no **navegador** (DartPad). Bloco 2 no **Android Studio** — começa a baixar **antes do intervalo** (internet boa), volta pronto.

Objetivos de aprendizagem

- **Comparar** os runtimes de RN e Flutter — arquitetura, forças e fraquezas
- **Construir** UI em Flutter compondo widgets (hands-on no DartPad)
- **Gerenciar estado** compartilhado com Riverpod (provider, sem prop drilling)
- **Entender** o modelo de concorrência do Flutter (**isolates** vs event loop)
- **Avaliar** quando ir nativo vs cross-platform com base em casos reais

Flutter em produção — quem usa (e por quê)

Nubank

Alibaba (Xianyu)

BMW

Google Pay

eBay Motors

Toyota

Por que apostaram em Flutter: uma base de código, **UI consistente** entre plataformas e iteração rápida. Hoje você vê **como** (widgets → estado → rendering) e, no fim, **quando** vale a pena.

 flutter.dev/showcase

Flutter em profundidade — Stack & Forças

Stack interno:

- **Dart** — linguagem com isolates (concorrência), AOT/JIT
- **Skia** → **Impeller** renderer (Impeller pré-compila shaders)
- **Widgets** — tudo é widget, composição radical
- **Riverpod** ou **Provider** para estado

Forças:

- UI consistente entre plataformas (renderiza própria)
- Performance gráfica estável — UI renderiza na GPU, sem cruzar bridge JS
- Hot reload rápido

"**Consistente**" na prática: um `Switch` fica **pixel-igual** no iOS e no Android (o Flutter desenha). No RN/nativo o mesmo toggle vira estilo iOS no iPhone e Material no Android.

Flutter em profundidade — Fraquezas

- Talent pool menor que RN
- UI pode parecer "não-nativa" — pra seguir o look da Apple precisa dos widgets **Cupertino** (o default é Material)
- Bundle size maior (engine Skia/Impeller + runtime Dart embarcados)
- Vantagem gráfica é relativa: a **New Architecture** do RN (JSI/Fabric) fechou boa parte do gap

Flutter — versões & o que mudou (2026)

Stable a cada ~3 meses, com o Dart embarcado. Hoje: Flutter 3.44 (mai/2026). A Atividade 3 fixa a 3.35.7 de propósito — reprodutibilidade do autograder.

Versão	Marco
3.29	Impeller vira default no iOS
3.38 (nov/25)	Impeller default no Android (API 29+) · Dart 3.10 (dot shorthands)
3.44 (mai/26)	Swift Package Manager estável · deep-link validator · Java 25/Gradle 9

Pra onde vai (roadmap 2026):

- 🎨 Impeller termina de substituir o Skia (remoção no Android)
- 🕸 WebAssembly a caminho de default na web (Dart→Wasm, sem JS · 2–3x em compute)
- ✨ Dart enxugando a sintaxe (menos boilerplate na widget tree)

📖 docs.flutter.dev/release/release-notes · blog.flutter.dev

Dart — sintaxe em 1 slide

```
var titulo = 'Matrix';           // tipo inferido (mutável)
final ano = 1999;                // imutável
double nota = 8.7;              // tipado explícito

String label(Filme f) => '${f.titulo} (${f.nota})'; // função (arrow)
String? sinopse;                // ? = pode ser null (null-safety)

class Filme {
  final String titulo;
  final double nota;
  Filme(this.titulo, this.nota); // construtor curto
}

Future<Filme> buscar(int id) async => Filme('Matrix', 8.7); // async/await · recebe o movieId
```

Primo do TypeScript: tipos, classes, `async/await` · `${...}` = interpolação · `?` = null-safety · `=>` = arrow.

Flutter — exemplo de widget

```
class CounterApp extends StatefulWidget {  
  const CounterApp({super.key});           // todo widget recebe um key  
  @override  
  State<CounterApp> createState() => _CounterAppState();  
}  
  
class _CounterAppState extends State<CounterApp> {  
  int _counter = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      body: Center(  
        child: Text('Count: $_counter', style: TextStyle(fontSize: 48)),  
      ),  
      floatingActionButton: FloatingActionButton(  
        onPressed: () => setState(() => _counter++),  
        child: Icon(Icons.add),  
      ),  
    );  
  }  
}
```

Tudo é widget. `setState` força rebuild — em apps reais, use Riverpod.

Tudo é widget — a "widget tree"

Scaffold (estrutura da tela)

AppBar → *title*: Text('Contador')

body: Center → **Column**

Text('Count: 0')

FloatingActionButton → Icon(add)

A UI é uma **árvore de widgets** que você **compõe**. Cada widget é uma classe; a tela inteira é função do estado.

StatelessWidget vs StatefulWidget

StatelessWidget

Só depende dos **parâmetros**. Não muda sozinho.

ex.: um `MovieCard(movie)` que só exhibe.

1 classe · só `build()`

StatefulWidget

Tem **estado mutável**. `setState()` dispara rebuild.

ex.: o contador, um toggle de favorito.

2 classes · estado vive na `State<...>` + `setState()`

estado muda → `setState()` → `build()` roda de novo → nova UI

Mesma ideia do React: **UI = f(estado)**. `setState` no Flutter ≈ `useState` no RN.

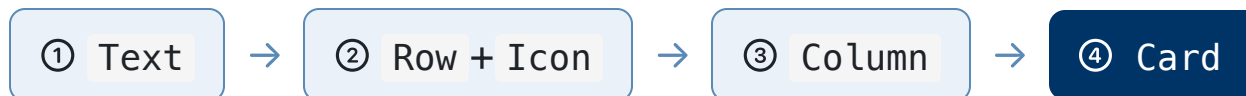
👉 **Qual escolher?** Comece sempre **Stateless** — mais simples e previsível. Promova pra **Stateful** só quando o widget precisa **lembrar de algo que muda** (um input, um toggle, uma animação). E quando o estado é **compartilhado por vários widgets**? Aí nem Stateful resolve bem — entra o **provider** (logo mais). 🎯

Vamos MONTAR o MovieCard (DartPad)

dartpad.dev — zero instalação. A meta: o **MovieCard** do app que vocês testaram — mas **construído por vocês**, peça por peça. *Tudo é widget = tudo é Lego.* 🧱



↑ onde a gente chega,
montando **4 peças** (widgets)



Regra do jogo: eu mostro a peça + os parâmetros · vocês trocam o `child:` e dão  **Run** · a próxima slide revela o encaixe.

Ponto de partida — cole no DartPad

O DartPad precisa de um **programa completo** (`main()`). Cole isto e dê  **Run** — deve aparecer o `Text('comece aqui')` na tela:

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(
  home: Scaffold(
    body: Center(child: /* 🖱️ suas peças entram aqui */ Text('comece aqui')),
  ),
));
```

A cada peça: troca o `child:` →  **Run**. (Se o botão **Reload** aparecer, ele faz *hot reload* — mais rápido — mas o **Run** sempre resolve.)

Peça 1 · **Text** (o título)

`Text` mostra uma string. O visual vem do parâmetro **style: TextStyle(...)** — `fontSize`, `fontWeight`, `color`.

```
Text('Matrix', style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold))
```

 **Sua vez:** ponha o título **Matrix** no lugar do `/* sua peça */` e dê **Reload**. Experimente: mude o `fontSize` e adicione `color: Colors.indigo` — veja a tela reagir.

Peça 2 · Row + Icon (a nota)

`Row(children: [...])` coloca widgets **lado a lado** (horizontal). `Icon` desenha um ícone — `Icon(Icons.star, color: Colors.amber, size: 18)`.

```
Row(children: [  
  Icon(Icons.star, color: Colors.amber, size: 18),  
  Text(' 8.7'),  
)
```

 **Sua vez:** monte a linha da nota  **8.7** — um `Icon` de estrela seguido de um `Text(' 8.7')`, dentro de um `Row`.

Peça 3 · Column (empilhar)

Column é o Row na **vertical**. Parâmetros úteis: `crossAxisAlignment` (alinha à esquerda) e `mainAxisSize: MainAxisSize.min` (ocupa só o necessário).

```
Column(  
  crossAxisAlignment: CrossAxisAlignment.start,  
  mainAxisSize: MainAxisSize.min,  
  children: [ /* título · nota · ano */ ],  
)
```

 **Sua vez:** empilhe num Column : o **título** (peça 1) + a **linha da nota** (peça 2) + o **ano** → `Text('1999 · Ficção', style: TextStyle(color: Colors.grey))` .

Peça final · Card + Padding = MovieCard 🎉

Card dá a moldura com elevação; Padding(EdgeInsets.all(16)) dá o respiro interno. Embrulhe o Column :

```
Card(child: Padding(
  padding: EdgeInsets.all(16),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Text('Matrix', style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold)),
      Row(children: [Icon(Icons.star, color: Colors.amber, size: 18), Text(' 8.7')]),
      Text('1999 · Ficção', style: TextStyle(color: Colors.grey)),
    ],
  ),
))
```

🎉 Vocês **construíram** o MovieCard — não copiaram. Mesmo card do app RN, agora em Flutter — e é exatamente o que você termina na **Atividade 3**.

Estado — e a dor de compartilhar

UI = f(estado). O `setState` reconstrói **1 widget**. Mas o estado de verdade é **compartilhado**: os favoritos aparecem no **card** e no **contador do header** ao mesmo tempo.

O problema: um widget lá no fundo precisa de um estado que mora longe. Passar por parâmetro, nível a nível, é o **prop drilling**. O Flutter tem um jeito nativo de evitar isso — o `InheritedWidget` (um widget que **expõe um valor pra toda a subárvore abaixo**, sem passar por parâmetro) — mas **escrever na mão é verboso**.



Prop drilling: carregar o estado mão-em-mão por toda a árvore. Não escala.

InheritedWidget — o jeito nativo (por que dói)

O Flutter já resolve "estado longe **sem** prop drilling" de fábrica: o **InheritedWidget** expõe um valor pra **toda a subárvore** abaixo; o descendente lê com `.of(context)` — sem passar por parâmetro.

```
class FavoritesScope extends InheritedWidget {  
  final Set<int> favorites;  
  const FavoritesScope({required this.favorites, required super.child});  
  
  static FavoritesScope of(BuildContext c) =>           // descendente lê assim  
    c.dependOnInheritedWidgetOfExactType<FavoritesScope>()!;  
  
  @override  
  bool updateShouldNotify(FavoritesScope old) => favorites != old.favorites; // quando re-renderiza  
}  
  
// SETUP: embrulha um ancestral na árvore (na mão) · descendente lê FavoritesScope.of(context)  
FavoritesScope(favorites: {1, 7}, child: const MaterialApp(home: Home()));
```

Funciona — mas é **muito boilerplate por estado** (classe + `of` + `updateShouldNotify` + um `StatefulWidget` por cima só pra **mutar**). Quase ninguém escreve na mão → é exatamente isso que o **provider** embrulha. →

2019 — Rémi Rousselet cria o **provider**

Rémi Rousselet empacota esse boilerplate do `InheritedWidget` num pacote ergonômico: o **provider**. Você **expõe** o estado no topo da árvore e qualquer descendente **lê** com `context.watch` — sem prop drilling, com pouco código. Ficou tão bom que o **Google** **passou a recomendar oficialmente** (Google I/O 2019).

```
// expõe o estado no topo da árvore  
ChangeNotifierProvider(create: (_) => Favorites(), child: App());  
  
// qualquer descendente lê pelo context — sem passar por parâmetro  
final favs = context.watch<Favorites>();
```

`InheritedWidget` (verboso)



provider · 2019 ✓ recomendado pelo Google

Guarde: o provider mora **dentro da árvore** (por isso depende do `BuildContext`) — é isso que o Riverpod vai virar do avesso. →

Provider — configurar (antes de usar)

① instale — no `pubspec.yaml`:

```
dependencies:  
  provider: ^6.1.2
```

② embrulhe o app **no root** — junto com o estado:

```
void main() => runApp(  
  ChangeNotifierProvider(  
    create: (_) => Favorites(),    // ➡ o estado vive AQUI, dentro da árvore  
    child: const MovieApp(),  
  ),  
);
```

O provider mora **dentro da árvore** → quem lê precisa do `BuildContext`. Feito isso, qualquer descendente usa `context.watch / read`. →

Provider — na prática

```
// 1. o estado — um ChangeNotifier
class Favorites extends ChangeNotifier {
  final _ids = <int>{};
  bool has(int id) => _ids.contains(id);
  void toggle(int id) { // favoritar/desfavoritar
    _ids.contains(id) ? _ids.remove(id) : _ids.add(id); // já é favorito? remove : adiciona
    notifyListeners(); // avisa quem ouve
  }
}

// 2. consome num widget (via BuildContext) — depois do setup do slide anterior
final favs = context.watch<Favorites>(); // ouve → re-render
context.read<Favorites>().toggle(movie.id); // ação (não ouve)
```

| **watch** re-renderiza quando muda; **read** só dispara a ação. Tudo depende do **BuildContext**.

Os novos problemas → nasce o Riverpod (2020)

Provider vive **dentro da árvore** → depende do `BuildContext`. Provider faltando = `ProviderNotFoundException` **em runtime**, não é type-safe, difícil testar. O mesmo Rémi reescreve como **Riverpod** (anagrama de "Provider" — recado: é o sucessor): tira o estado **pra fora da árvore**.

Provider — dentro da árvore

precisa de `BuildContext` · erro em **runtime**

Riverpod — fora da árvore

global · **compile-safe** · testável

```
// global, fora da árvore – o compilador garante que existe
final favoritesProvider = NotifierProvider<Favorites, List<int>>(Favorites.new);
final favs = ref.watch(favoritesProvider); // ConsumerWidget + ref – sem context
```

PROVIDER ⇔ RIVERPOD (anagrama). Riverpod é a recomendação atual; Provider segue comum em base legada.

 riverpod.dev · pub.dev/packages/provider

O mesmo bug: Provider 🌟 vs Riverpod ✅

Você esqueceu de registrar o provider. O que cada um faz:

Provider — compila, mas **crasha em runtime**

```
// sem ChangeNotifierProvider acima...
final favs = context.watch<Favorites>();
// 🌟 ProviderNotFoundException
//    - estoura em RUNTIME, no device
```

Riverpod — o compilador **te barra antes**

```
// favoritesProvider: global + tipado
final favs = ref.watch(favoritesProvider);
// ✅ se não existisse, NEM COMPILA
//    - o editor acusa, nunca chega no app
```

Provider erra **tarde** (no device, na frente do usuário); Riverpod erra **cedo** (no seu editor). Errar cedo é mais barato.



Riverpod — configurar (antes de usar)

① instale — no `pubspec.yaml`:

```
dependencies:  
  flutter_riverpod: ^2.5.1
```

② embrulhe o app **no root** — uma vez só, no `main`:

```
void main() => runApp(  
  const ProviderScope(child: MovieApp()), // 👉 liga o Riverpod na árvore inteira  
);
```

Sem o `ProviderScope` no topo, **nenhum** `ref.watch` funciona. Feito isso, é só **declarar** providers globais e **consumir** — sem mais setup. →

Riverpod — na prática

```
// 1. estado (Notifier) + provider GLOBAL (fora da árvore)
class Favorites extends Notifier<Set<int>> {
  @override
  Set<int> build() => {};
  void toggle(int id) => // favoritar/desfavoritar
    state = state.contains(id) ? ({...state}..remove(id)) : {...state, id}; // já é? remove : adiciona
}
final favoritesProvider =
  NotifierProvider<Favorites, Set<int>>(Favorites.new);

// 2. consome num ConsumerWidget (SEM BuildContext)
class MovieCard extends ConsumerWidget { // era StatelessWidget
  Widget build(context, ref) { // o `ref` entra aqui (2º param) – nenhum import a mais
    final isFav = ref.watch(favoritesProvider).contains(movie.id); // ouve
    return IconButton(
      onPressed: () => ref.read(favoritesProvider.notifier).toggle(movie.id), // ação
      icon: Icon(isFav ? Icons.favorite : Icons.favorite_border),
    );
  }
}
```

`ref.watch` ouve · `ref.read(...notifier)` dispara ação. **Compile-safe, sem `BuildContext`** — é o que você usa na Atividade 3.

Provider × Riverpod — o que muda

Olhando o código, mudam só 3 coisas — o resto da lógica é igual:

- ① **pacote** — `provider` → `flutter_riverpod`
- ② **setup** — `ChangeNotifierProvider(...)` na árvore → `ProviderScope(child: App())` na raiz
- ③ **ler/agir** — `context.watch/read<X>()` → `ref.watch/read(xProvider)`

Mesma **API mental** (`watch` / `read`); o Riverpod só tira o estado **pra fora da árvore** → some o `BuildContext`, vira compile-safe e testável.

Provider × Riverpod — tabela de referência

	Provider	Riverpod
Pacote (pubspec.yaml)	<code>provider</code>	<code>flutter_riverpod</code>
Setup na raiz	<code>ChangeNotifierProvider(...)</code> na árvore	<code>ProviderScope(child: App())</code>
Declara o estado	<code>extends ChangeNotifier</code>	<code>extends Notifier<T></code>
Lê o estado	<code>context.watch<X>()</code>	<code>ref.watch(xProvider)</code>
Dispara ação	<code>context.read<X>().toggle()</code>	<code>ref.read(xProvider.notifier).toggle()</code>
Widget que consome	qualquer + <code>BuildContext</code>	<code>ConsumerWidget</code> (recebe <code>ref</code>)
Provider faltando	erro em runtime ⚠️	erro de compilação ✅

Setup — Android Studio + Flutter (começa AGORA)

A internet aqui aguenta — **dispara o download antes do café** e volta com tudo pronto:

1. **Android Studio** → `developer.android.com/studio`
2. **Flutter SDK** → `docs.flutter.dev/get-started/install`
3. Plugin **Flutter** no Android Studio (Settings → Plugins)
4. Confere: `flutter doctor`

No Bloco 2 abrimos o **projeto da Atividade 3** (já com o fork atualizado):

```
cd exercicios/03-flutter-ui-estado/pratica # ➡ o projeto fica AQUI
flutter pub get && flutter run -d chrome   # navegador, sem emulador
```



Deixa o download rolando no intervalo.

Atualize seu fork — pega a Atividade 3

Antes de abrir o projeto: traga os commits do prof (a **Atividade 3** é arquivo novo) pro seu fork — 2 min:

```
# 1x só (se ainda não tem o upstream): aponta pro repo do prof
git remote add upstream https://github.com/jacksonsmith/puc-iec-mobile-multiplataforma.git

# a cada aula: puxa o que há de novo
git fetch upstream
git checkout main
git merge upstream/main      # traz exercicios/03-flutter-ui-estado/ pro seu main
git push                    # atualiza seu fork no GitHub
```

Sem isso, a pasta da **Atividade 3** não aparece no seu fork. Tem mudança local? `git stash` antes. Guia completo: `COMO_ENTREGAR.md` (raiz do repo).

Intervalo — 30min

Volta às :__

⬇️ **Deixe o Android Studio + Flutter baixando** — é pra isso que o intervalo serve hoje.

Esticar perna · café/água · rode `flutter doctor` quando instalar.

| No Bloco 2 a gente vai pro **Android Studio** abrir e rodar o projeto da Atividade 3.

Como o Flutter desenha — o caminho até o pixel

O Flutter **não usa componente nativo** — ele **desenha cada pixel** ele mesmo. O caminho de um widget até a tela:



Shader = um programinha que diz *a cor de cada pixel*, rodando nos **milhares de núcleos da GPU** (vem do mundo de jogos/OpenGL). Gradiente, sombra, blur, cantos arredondados — **cada efeito é um shader**.

No **RN/nativo** isso **não existe** — quem desenha é o componente do SO. Desenhar tudo sozinho dá o **pixel-igual** entre plataformas... e cria o problema do próximo slide. →

De onde vem o *jank* — Skia compila em runtime

O shader precisa ser ***compilado*** (código tipo-C → instruções da GPU). O Skia — motor 2D do Google, o mesmo do Chrome e do Android — faz isso **on-demand**: compila o shader **só na 1ª vez que aquele efeito aparece**, em runtime, no meio do frame.

O problema — "shader compilation jank": a primeira vez que um gradiente ou animação nova entra em cena, a GPU **para pra compilar o shader naquele instante** → o frame engasca. É como o restaurante só começar a cozinhar o prato quando o 1º cliente pede: todo mundo espera. Pior no iOS (compilar no Metal é caro).

A virada: Impeller pré-compila tudo

A ideia do Impeller (novo motor do Flutter, 2023): em vez de compilar shader em runtime, **pré-compila TODOS em build time**. Quando o app roda, **não há nada pra compilar** — o 1º frame é tão liso quanto o centésimo. (Voltando à analogia: o prato já sai pronto da cozinha.)

- **Some o jank de primeira vez** — o custo saiu do caminho crítico pro build
- Fala **direto com Metal (iOS) e Vulkan (Android)** — APIs de GPU modernas, sem a camada antiga do Skia
- Só dá pra pré-compilar porque o conjunto de efeitos do Flutter é **fixo e conhecido**

Estado em 2026: Impeller é **default no iOS e no Android (API 29+)**; em Android antigo cai no OpenGL. No iOS o Skia foi descontinuado.

Pro aluno: é o tipo de detalhe de runtime que **não existe no RN** (lá quem desenha é o componente nativo) — bom argumento no comparativo.

 Fonte: docs.flutter.dev/perf/impeller

Comparativo de runtime

Aspecto	 React Native (Hermes)	 Flutter (Dart)
 Engine	Hermes (bytecode)	Dart VM (AOT)
 Linguagem	TS / JS	Dart
 UI	Componentes nativos	Self-rendered (Skia/Impeller)
 Hot reload	Sim (rápido)	Sim (instantâneo)
 Bundle	~25–40 MB	~40–60 MB
 Concorrência	Event loop (1 thread)	Isolates (heap próprio)
 Talent	Alto	Médio

 A diferença que importa: RN usa **componentes nativos** (parece nativo de graça, mas varia por plataforma); Flutter **desenha cada pixel** (consistência total entre OS, com o jank de shaders resolvido pelo Impeller).

 Bundle/runtime: docs.flutter.dev/perf/app-size

Concorrência — 3 jeitos de não travar a UI

■ RN / JS — event loop

1 thread só. `async/await` intercala tarefas nessa mesma thread (concorrência de I/O) — não roda nada **em paralelo na CPU**. Loop pesado → **trava a UI**.

■ Flutter / Dart — isolates

Várias threads, memória isolada. Cada isolate tem seu **heap** — **não compartilham** estado (sem race), e trocam dados **por mensagem** (paga serialização).

■ Nativo (Kotlin/Swift)

Threads reais do SO → **paralelismo** de verdade nos núcleos da CPU. A **coroutine suspende sem bloquear** a thread. Regra: trabalho pesado **fora da UI thread**.

Isolate ≠ thread: thread compartilha memória (precisa de lock, arrisca race condition); isolate **não compartilha** (sem race, mas paga a serialização da mensagem).

Concorrência na prática

RN / JS — `async` é I/O concorrente, não CPU paralelo:

```
const [a, b] = await Promise.all([fetchA(), fetchB()]); // I/O sobreposto  
// CPU pesado? quebre em pedaços ou mande pro nativo – nunca trave o event loop
```

Flutter / Dart — `compute()` joga a função pra outro isolate:

```
final filmes = parseFilmes(jsonGigante); // ❌ na main: 5.000 filmes congelam a rolagem  
final filmes = await compute(parseFilmes, jsonGigante); // ✅ outro isolate → parse em paralelo, UI lisa
```

Nativo (Kotlin/Swift) — threads reais do SO + `coroutines` / `async` que **suspendem sem bloquear**. (É o *Kotlin* que vocês veem no **bridging da Aula 4** — por isso aparece aqui; não precisa decorar agora.)

A regra comum às stacks: **nunca segure a thread/loop da UI** — o que muda é como cada uma manda o trabalho pesado pra longe.

📖 dart.dev/language/concurrency · docs.flutter.dev/perf/isolates

Decisão: nativo vs cross-platform

Vai NATIVO se...

- Performance gráfica intensiva (jogos, AR, vídeo)
- Hardware profundo (NFC avançado, BLE Mesh, sensores)
- Time com 5+ engenheiros nativos por plataforma
- Compliance exige (bancário/governo)

Vai CROSS-PLATFORM se...

- Time multidisciplinar (web + mobile)
- ROI cross-OS importa (manutenção compartilhada)
- Velocidade de iteração > performance ultra
- App é CRUD + integração (maioria dos casos)

Não há "verdade universal" — é **trade-off contextual**. É a decisão que você, como arquiteto, precisa saber **defender com dados** (não "depende").

Testar Flutter — unit × widget

`flutter test` roda tudo na pasta `test/`. Dois tipos:

unit test

testa **lógica pura** (um provider, uma função). Rápido, sem tela — `ProviderContainer`.

widget test

testa **a UI** com `WidgetTester`: `pump`, `tap`, `find.text`.

Pirâmide: muitos **unit** (baratos) + alguns **widget**. Na Atividade 3, o **gate** é `flutter test`.

Testando o estado (ProviderContainer)

```
test('favoritos: toggle e clear', () {  
  final c = ProviderContainer();           // mini-app só pro teste  
  addTearDown(c.dispose);  
  
  expect(c.read(favoritesProvider), isEmpty);  
  c.read(favoritesProvider.notifier).toggle(1);  
  expect(c.read(favoritesProvider).contains(1), isTrue);  
});
```

read lê o estado · .notifier chama a ação. É o Ex3 da Atividade — você escreve um teste assim. (Na UI: `WidgetTester + tester.tap + find.text('❤️ 1') .)`

Atividade 3 (da disciplina) — App Flutter: UI + Estado + Testes (15 pts)

📌 É a 3ª atividade da disciplina (1 por aula) — não o 3º exercício de hoje. São 3 exercícios que, juntos, formam a Atividade 3.

Entrega: até 23/06. Abrimos juntos no **Android Studio**, você termina em casa.

- **Ex1 · UI** — compor o `MovieCard` (lista de filmes)
- **Ex2 · Estado (Riverpod)** — favoritar + contador no header + **limpar** (uma fonte só, sem prop drilling)
- **Ex3 · Testes** — **você escreve** um teste do provider (`flutter test`)

Scaffold + rubrica no enunciado. **Bridging nativo entra na Aula 4** (KMP + Platform Channel).

Leituras & Referências

Obrigatórias (pré-Aula 4):

- Windmill, E. — *Flutter in Action* (Manning), cap 1–3
- Airbnb Engineering — *Sunsetting React Native* (5 posts · [material-de-apoio/aula-01/](#))
- Joorabchi, Mesbah, Kruchten (2013) — *Real Challenges in Mobile App Development*, ESEM
- JetBrains — *Kotlin Multiplatform Architecture* (prep do 3-way da Aula 4)
- W3C — *Service Worker Specification* (prep Aula 4)

Fontes da aula (oficiais):

- Flutter rendering: [docs.flutter.dev/perf/impeller](#) · bundle: [/perf/app-size](#)
- Concorrência: [dart.dev/language/concurrency](#) · [docs.flutter.dev/perf/isolates](#)
- Estado: [riverpod.dev](#) · [pub.dev/packages/provider](#)
- Bridging: [reactnative.dev](#) (Turbo Native Modules) · [docs.expo.dev/modules](#)
- Caso Discord: [discord.com/blog/why-discord-is-sticking-with-react-native](#)



Pacote completo em [material-de-apoio/aula-03/README.md](#) .

Próxima aula (24/06)

Aula 4 — Native interop: KMP + Native Bridging

Quando o cross-platform encosta no nativo: **KMP** (compartilha lógica) + **bridging** (Platform Channel / TurboModule). O exercício faz a bridge. (*PWA vai pra Aula 5.*)

Pré-requisito:

- Atividade 3 entregue (23/06)
- Ler W3C Service Worker spec (parte 1 e 2)
- Ler Jake Archibald — *The Offline Cookbook*

Obrigado!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Próxima quarta, 24/06, 19:00